SEGURANÇA DE DADOS • CC-UENF

Estudo e Análise de Algoritmos KDF

Análise comparativa das funções de derivação de chave: PBKDF2, bcrypt e scrypt

Disciplina: Segurança de Dados **Professor:** Prof. Ausberto S. Castro Vera **Tópico:** Funções Hash & Armazenamento Seguro

Resumo Executivo: Funções de hash tradicionais (como MD5 e SHA-256) foram projetadas para velocidade de cálculo, o que as torna vulneráveis a ataques de força bruta paralelos quando aplicadas ao armazenamento de senhas. Para mitigar esse risco, utilizam-se Funções de Derivação de Chave baseadas em Senha (KDFs), que aplicam fator de custo para retardar a computação dos hashes.

1. Introdução às Funções KDF

O armazenamento seguro de senhas exige que a transformação do texto limpo em hash seja computacionalmente custosa. Diferente das funções hash de uso geral, algoritmos como **PBKDF2**, **bcrypt** e **scrypt** inserem atrasos intencionais através do aumento da complexidade de processamento e/ou do consumo de memória. Essa abordagem inviabiliza ataques massivos por meio de *Rainbow Tables* e desacelera significativamente tentativas via hardware especializado (GPUs, FPGAs e ASICs).

2. Análise Detalhada dos Algoritmos

PBKDF2 (Password-Based Key Derivation Function 2) FIPS/NIST CPU Bound

Padronizado pelo RFC 2898 e aprovado pelo NIST, o PBKDF2 aplica repetidamente uma função pseudorrandômica (como HMAC-SHA256) juntamente com um *salt* a um texto simples. O seu parâmetro principal é o **número de iterações**. Embora eficaz no controle de custo em CPUs convencionais, apresenta fragilidade contra aceleração por hardware (GPUs e ASICs), pois possui baixíssima dependência de memória.

bcrypt CPU Bound Padrão Indústria

Baseado no algoritmo de cifragem Blowfish (especificamente o esquema *Eksblowfish*), o bcrypt introduz um fator de custo exponencial (2^{custo} iterações). Ele inicializa o seu estado interno através de um processo intensivo em trocas de chaves, fazendo uso moderado da memória local. Essa característica limita sensivelmente a eficiência de ataques massivamente paralelos em placas gráficas (GPUs).

scrypt Memory-Hard Alta Proteção

Projetado especificamente para impedir a criação de circuitos integrados dedicados (ASICs), o scrypt é uma função *memory-hard*. Ele exige não apenas tempo de CPU, mas também uma quantidade considerável e configurável de memória RAM durante o cálculo. Como chips customizados possuem limitações físicas de memória integrada, atacar hashes scrypt em larga escala torna-se extremamente dispendioso.

3. Quadro Comparativo Técnico

A tabela a seguir consolida as características estruturais, vetores de defesa e o nível de proteção oferecido por cada algoritmo contra diferentes vetores de ataque:

Algoritmo	Fator de Custo	Uso de Memória	Resistência a GPUs/ASICs	Cenário de Uso Recomendado
PBKDF2	Iterações em CPU	Muito Baixo (~negligenciável)	Baixa (Vulnerável à paralelização)	Sistemas legados, requisitos rígidos de conformidade FIPS/NIST.
bcrypt	Rondas (2^{cost})	Moderado (~4 KB de cache)	Média / Alta (Eficaz contra GPUs)	Aplicações web modernas e sistemas corporativos gerais.
scrypt	CPU + Memória (N, r, p)	Configurável (Avançado/Elevado)	Muito Alta (Invabiliza ASICs)	Sistemas de alta segurança e carteiras de criptomoedas.

4. Mecanismos de Proteção e Fatores de Configuração

- **Salt Obrigatório:** Todos os três algoritmos exigem o uso de um *salt* aleatório e exclusivo para cada usuário. Isso anula completamente a eficácia de atalhos pré-computados (como *Rainbow Tables*).
- **Ajuste de Fator de Custo (Work Factor):** À medida que o hardware do atacante evolui com o passar dos anos, os parâmetros de custo (iterações no PBKDF2/scrypt, ou *cost factor* no bcrypt) podem e devem ser incrementados sem alterar a estrutura do banco de dados.
- **Complexidade de Hardware:** Algoritmos *memory-hard* como o **scrypt** forçam cada unidade de processamento paralela do atacante a alocar memória individual, o que limita drasticamente o número de núcleos que podem operar simultaneamente em uma GPU ou ASIC.

5. Conclusões e Recomendações

Para o desenvolvimento de novos sistemas, a adoção de algoritmos com requisitos de memória é altamente recomendada. O **scrypt** e o **bcrypt** figuram como as escolhas mais sólidas para proteção de senhas em repouso. O **PBKDF2** deve ser priorizado apenas em cenários onde normas governamentais exigem expressamente padrões como FIPS-140.

Em termos de evolução tecnológica, vale ressaltar que o algoritmo *Argon2* (vencedor do *Password Hashing Competition*) e o *scrypt* representam o estado da arte no combate a ataques por hardware acelerado.